

Hook

Estás usando ChatGPT. Le explicás tu proyecto, le pedís que siga ciertas reglas, y funciona bárbaro. Van y vienen mensajes. Todo bien. [pausa] Hasta que en el mensaje 15... te responde cualquier cosa. Como si nunca le hubieras dicho nada. [pausa] ¿Qué pasó?

Máquina de Predicción

Para entender por qué pasa esto, necesitamos entender un poco cómo funciona un LLM por dentro. Un Large Language Model no “entiende” nada. Es una máquina de predicción de texto: dado todo lo anterior, predice qué palabra viene después. Pero esa predicción es mucho más sofisticada de lo que parece. El año pasado, Anthropic — la empresa detrás de Claude — le hizo reverse engineering a uno de sus modelos y publicó lo que encontró adentro. Y es fascinante. Primero: no procesa paso a paso como haríamos nosotros. Evalúa múltiples caminos en paralelo y “vota” entre ellos. Segundo: planifica hacia adelante. Puede elegir una palabra que va a decir al final de una oración antes de escribir el principio de esa oración. ¿Cómo lo hace? Con circuitos internos.

Circuitos Internos

Esta imagen viene del paper de Anthropic. Es un ejemplo real de cómo razona el modelo. Le preguntás: “¿Cuál es la capital del estado donde está Dallas?” Y el modelo no busca la respuesta directamente. Encadena representaciones internas: activa “Dallas”, eso activa “Texas”, eso se combina con “capital”, y llega a “Austin”. Cada uno de estos nodos es lo que en interpretabilidad llaman un feature — una representación interna, no confundir con feature de un dataset.

Atención

Para generar cada palabra, el modelo tiene que decidir a qué parte de todo el texto prestarle atención. Esto se llama, literalmente, el mecanismo de atención. Cuando el contexto es corto esto funciona bien. Pero la atención es un recurso finito. Se reparte entre todos los tokens del contexto. Y cuando el contexto crece, las instrucciones que le diste al principio tienen que competir con todo lo que vino después. ¿Qué pasa cuando hay un millón de tokens compitiendo por atención?

Ventana de Contexto

Un token es la unidad mínima que procesa el modelo. Una palabra es más o menos uno o dos tokens. Una página son unos 500 tokens. La ventana de contexto es todo lo que el modelo puede “ver” al responder: el system prompt, toda la conversación, los archivos adjuntos. Si algo no está en la ventana, el modelo no lo sabe. No tiene memoria aparte. En 2023, la ventana más grande era de 16 mil tokens. Hoy estamos en un millón. Eso es dos mil páginas. Más ventana debería ser mejor, ¿no?

System Prompt y Guardrails

El system prompt son las instrucciones que recibe el modelo antes de hablar con el usuario. Cosas como: “respondé en español”, “sos un tutor amable”, “no generes contenido dañino”, “usá tal herramienta para tal tarea”. A las instrucciones de seguridad les decimos guardrails. Y acá hay algo importante: los guardrails no son un módulo especial. El modelo no tiene un botón de “seguridad”. Son instrucciones como cualquier otra. La seguridad es un pathway — un camino de activaciones. No es un interruptor.

La Evidencia

Laban y colegas en 2025 midieron una caída del 39% en tareas conversacionales de varios turnos. El equipo de Meta con Multi-IF mostró que la precisión baja de 87.7% a 70.7% en solo 3 turnos. Chroma Research testeó 18 modelos frontera — y los 18 muestran el mismo patrón. Y Mu y colegas mostraron que cuando le ponés más de 50 reglas al system prompt, la adherencia se acerca a cero. No es un bug de un modelo. Le pasa a todos.

La Curva U

¿Y por qué pasa? En parte lo explica “Lost in the Middle” de Liu et al. Cuando medís la precisión según dónde está la información en el contexto, aparece una curva U. Lo del principio lo atiende bien. Lo del final, también. Pero lo del medio se pierde. Esto se midió en tareas de recuperación, no directamente en instrucciones. Pero sugiere que la atención no es uniforme.

¿Por qué importa?

Plata: las empresas procesan miles de millones de tokens por día. Cada token de reminder cuesta. Refuerzo ineficiente a esa escala es mucha plata. Seguridad: nuestra hipótesis es que los guardrails podrían decaer más rápido que las instrucciones de formato. Si es así, los reminders uniformes sub-invierten en seguridad. Ejemplo: le pedís que solo cite fuentes reales. 10 turnos después, te inventa un paper que no existe.

¿Qué se Hace Hoy?

Repetir todo cada turno: satura. Duplicar el prompt: 2× costo. No hacer nada: 39% de caída. Jerarquía de prioridad: no aborda el decaimiento gradual. Ninguna es adaptativa. Ninguna decide qué reforzar ni cuándo.

La Pregunta Clave

Si sabemos que el modelo olvida... y sospechamos que cada instrucción decae distinto... ¿por qué reforzamos todo por igual? [pausa] ¿Y si pudiéramos reforzar solo lo que se está por olvidar?

Bayesiano vs Clásico

El enfoque clásico es binario. ¿Cumple? Sí o no. No distingue entre 90% y 51%. El enfoque bayesiano: mantenemos un posterior para cada instrucción. Cada observación actualiza la estimación — verosimilitud por prior. Entre turnos, el posterior decae. Podemos anticipar el olvido antes de que pase.

BKT

Bayesian Knowledge Tracing se usa hace 30 años en educación. Estima la probabilidad de que un estudiante domine un concepto, turno a turno. La inversión conceptual: ¿qué pasa si el “estudiante” es el LLM y los “conceptos” son sus instrucciones?

Modelando el Olvido

Para cada instrucción i en el turno t , la probabilidad de que siga siendo cumplida es un Bayes update clásico. $\gamma < 1$ controla cuánto decae entre turnos. Si tenés un presupuesto fijo de B tokens para reminders, reforzás solo las instrucciones con menor probabilidad de compliance. Nadie aplicó BKT a compliance de LLMs, ni midió curvas de olvido por tipo de instrucción, ni optimizó qué reforzar con un presupuesto limitado de tokens. Eso es lo que propongo investigar.